Sesión 13

Despliegue y Alta Disponibilidad

Carlos César Sánchez Coronel

BASES DE DATOS

2026

Contenido general

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Objetivos de la sesión

- Comprender la importancia de la contenedorización para bases de datos.
- Aprender a desplegar SGBD en Docker y configurar persistencia.
- Conocer los conceptos básicos de Kubernetes aplicados a bases de datos.
- Diferenciar estrategias de alta disponibilidad: réplicas de lectura, failover, clustering.
- Configurar replicación en motores populares (PostgreSQL, MySQL).
- Entender el balanceo de carga para bases de datos.
- Analizar opciones en la nube (RDS, Cloud SQL) y comparar con on-premise.
- Planificar y ejecutar migraciones de bases de datos.
- Realizar ejercicios prácticos paso a paso.

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

¿Por qué contenedores?

- Entornos consistentes (desarrollo, prueba, producción).
- Rápido aprovisionamiento y destrucción.
- Aislamiento de recursos.
- Integración con CI/CD.

Ejemplo: PostgreSQL en Docker

```
# Descargar imagen
docker pull postgres:15

# Ejecutar contenedor con persistencia
docker run -d \
  --name mi-postgres \
  -e POSTGRES_PASSWORD=mi_password \
  -e POSTGRES_USER=usuario \
  -e POSTGRES_DB=mi_db \
  -v pgdata:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:15
```

- `-v pgdata:/var/lib/postgresql/data`: volumen nombrado para persistencia.
- Variables de entorno configuran usuario, contraseña y base de datos inicial.

Volúmenes en Docker

Volumen nombrado Gestionado por Docker, ubicado en `/var/lib/docker/volumes/`. Recomendado para producción.

Bind mount Monta un directorio del host. Útil en desarrollo.

Comandos

```
docker volume create pgdata
docker run -v pgdata:/var/lib/postgresql/data ...
```

Redes en Docker

Para comunicación entre contenedores:

```
docker network create mi-red
docker run -d --name postgres --network mi-red -e POSTGRES_PASSWORD=... postgres
docker run -d --name app --network mi-red -p 8080:8080 mi-app
```

La aplicación accede a la BD usando el hostname `postgres`.

Consideraciones de Rendimiento en Producción

- Usar volúmenes rápidos (SSD).
- Limitar recursos con `-memory` y `-cpus`.
- Ajustar parámetros del kernel (ej. `shared_buffers`).
- Configurar rotación de logs.

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Kubernetes para Bases de Datos

- Despliegues declarativos (Deployments, StatefulSets).
- Auto-escalado.
- Balanceo de carga interno (Services).
- Auto-reparación (reinicio de contenedores fallidos).

StatefulSet vs Deployment

Deployment Para aplicaciones sin estado. Pods intercambiables.

StatefulSet Para aplicaciones con estado. Identidades estables (nombres como `postgres-0`), orden en despliegue/escalado, y almacenamiento persistente dedicado.

StatefulSet es el adecuado para bases de datos.

Ejemplo: StatefulSet para PostgreSQL

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:15
        env:
        - name: POSTGRES_PASSWORD
          value: "mi_password"
        - name: POSTGRES_USER
          value: "usuario"
        - name: POSTGRES_DB
          value: "mi_db"
        ports:
        - containerPort: 5432
```

Service para acceder a PostgreSQL

```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
  clusterIP: None  # headless para descubrimiento
```

Operadores para Bases de Datos en Kubernetes

Extienden Kubernetes para tareas complejas: backups, failover, upgrades.

- Crunchy PostgreSQL Operator
- Zalando PostgreSQL Operator
- Oracle MySQL Operator

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Conceptos: RTO y RPO

RTO Recovery Time Objective: tiempo máximo para restaurar servicio.

RPO Recovery Point Objective: pérdida máxima de datos aceptable.

Las estrategias HA buscan minimizar RTO y RPO.

Réplicas de Lectura

- Copias de la base principal que reciben cambios y aceptan lecturas.
- Beneficios: escalabilidad de lecturas, aislamiento de cargas, mayor disponibilidad.
- Implementaciones: PostgreSQL streaming replication, MySQL replication, MongoDB replica sets.

Failover

Conmutación a nodo secundario ante fallo del primario.

- **Manual** Administrador promueve la réplica.
- **Automático** Sistema de monitoreo ejecuta promoción (Patroni, repmgr, Orchestrator).

Riesgo de split-brain: usar quorum (etcd, ZooKeeper) y fencing.

Clustering

Activo-Pasivo Un nodo activo, otros en espera. Ej: PostgreSQL con Patroni.

Activo-Activo Todos los nodos aceptan escrituras, con resolución de conflictos. Ej: MySQL Cluster, Oracle RAC, Cassandra.

Balanceo de Carga

Distribuye tráfico entre varios servidores.

- **HAProxy**: balanceo TCP genérico.
- **ProxySQL**: balanceo y enrutamiento para MySQL.
- **pgpool-II**: pooling y balanceo para PostgreSQL.

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

PostgreSQL: Streaming Replication

Primario

```
CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD 'pass';
```

En postgresql.conf:

```
wal_level = replica
max_wal_senders = 5
wal_keep_size = 1GB
```

En pg_hba.conf:

```
host replication replicator 192.168.1.0/24 md5
```

Secundario

```
pg_basebackup -h primario -D /var/lib/postgresql/data -U replicator -v -P --wal-method=stream
touch /var/lib/postgresql/data/standby.signal
echo "primary_conninfo = 'host=primario user=replicator password=pass'" >> postgresql.auto.conf
```

MySQL: Group Replication

- Replicación multi-maestro con detección automática de fallos.
- Configuración mediante MySQL Shell (InnoDB Cluster).
- Proporciona consistencia fuerte y failover automático.

MongoDB: Replica Sets

- Grupo de servidores mongod con un primario y varios secundarios.
- Failover automático: si el primario falla, los secundarios eligen uno nuevo.
- Configuración: iniciar cada nodo con `-replSet`, luego `rs.initiate()` y `rs.add()`.

Oracle: Data Guard

- Mantiene bases de datos standby (físicas o lógicas).
- Puede ser síncrona (protección máxima) o asíncrona.
- Fast-Start Failover para conmutación automática.

SQL Server: Always On Availability Groups

- Grupos de disponibilidad con réplicas síncronas o asíncronas.
- Hasta 9 réplicas, failover automático con cluster de Windows.
- Réplicas legibles.

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

HAProxy para PostgreSQL

Configuración en `/etc/haproxy/haproxy.cfg`:

```
global
    log stdout format raw local0
defaults
    log global
    option tcplog
    timeout connect 5s
    timeout client 1m
    timeout server 1m
frontend pgsql
    bind *:5432
    default_backend pgsql_backend
backend pgsql_backend
    balance roundrobin
    option pgsql-check user haproxy
    server pg1 192.168.1.10:5432 check
    server pg2 192.168.1.11:5432 check
```

Crear usuario `haproxy` en PostgreSQL para health checks.

ProxySQL para MySQL

- Enrutamiento de consultas (lecturas a réplicas, escrituras al primario).
- Pooling de conexiones, caché de consultas, control de tráfico.
- Configuración mediante tablas en memoria.

pgpool-II para PostgreSQL

- Middleware con pooling de conexiones, balanceo de carga, replicación y failover.
- Se sitúa entre la aplicación y los servidores PostgreSQL.

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Servicios Gestionados (PaaS)

AWS RDS Soporta múltiples motores, replicación, backups, failover multi-AZ.

Google Cloud SQL PostgreSQL, MySQL, SQL Server con alta disponibilidad.

Azure SQL Database Base de datos como servicio escalable.

Ventajas: menor overhead, escalado sencillo, HA integrada.
Desventajas: menos control, vendor lock-in, costos variables.

On-Premise

Ventajas: control total, cumplimiento normativo, costos fijos.
Desventajas: inversión inicial alta, mantenimiento complejo, escalado limitado.

Arquitectura Híbrida

Combinación de on-premise y nube. Ej: datos sensibles on-premise, data lake en la nube para análisis, replicación para disaster recovery.

Migración a la Nube: Ejemplo con pg_dump

```
# Dump local
pg_dump -U usuario -h localhost mi_db > mi_db.sql

# Restaurar en RDS (previamente crear instancia)
psql -U usuario -h mi-instance.rds.amazonaws.com -d mi_db < mi_db.sql
```

Para migración sin downtime: usar AWS DMS con replicación continua (CDC).

AWS Database Migration Service (DMS)

- Servicio gestionado para migraciones con mínimo downtime.
- Soporta replicación continua desde el origen.
- Ideal para grandes volúmenes o migraciones heterogéneas (ej. Oracle a PostgreSQL).

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Ejercicio 1: PostgreSQL con Docker y persistencia

Enunciado: Crear contenedor PostgreSQL con volumen persistente, crear tabla, insertar datos, detener y eliminar contenedor, luego crear nuevo con mismo volumen y verificar persistencia.

Solución

```
docker volume create pgdata
docker run -d --name postgres1 -e POSTGRES_PASSWORD=admin -e POSTGRES_USER=admin -e POSTGRES_DB=
    testdb -v pgdata:/var/lib/postgresql/data -p 5432:5432 postgres:15
docker exec -it postgres1 psql -U admin -d testdb
# En psql:
CREATE TABLE personas (id SERIAL, nombre TEXT);
INSERT INTO personas (nombre) VALUES ('Ana'), ('Luis');
SELECT * FROM personas;
\q
docker stop postgres1 && docker rm postgres1
docker run -d --name postgres2 -e POSTGRES_PASSWORD=admin -e POSTGRES_USER=admin -e POSTGRES_DB=
    testdb -v pgdata:/var/lib/postgresql/data -p 5432:5432 postgres:15
docker exec -it postgres2 psql -U admin -d testdb -c "SELECT * FROM personas;"
```

Ejercicio 2: Configurar replicación PostgreSQL con Docker

Enunciado: Levantar dos contenedores PostgreSQL (primario y réplica) en una misma máquina y configurar streaming replication asíncrona.

Solución (resumida)

```
docker network create pg-net
# Primario
docker run -d --name pg-primary --network pg-net -e POSTGRES_PASSWORD=admin -e POSTGRES_USER=
    admin -e POSTGRES_DB=testdb -v pg-primary-data:/var/lib/postgresql/data postgres:15
# Configurar primario (wal_level, max_wal_senders, pg_hba) y crear usuario replicator
# Réplica: hacer basebackup y configurar standby
```

Ver detalles en el solucionario.

Ejercicio 3: Failover manual en PostgreSQL

Enunciado: Simular caída del primario y promover la réplica.

Solución

```
docker stop pg-primary
docker exec -it pg-replica pg_ctl promote -D /var/lib/postgresql/data
# Ahora pg-replica es primario
```

Ejercicio 4: Migrar MySQL local a AWS RDS (simulado)

Enunciado: Migrar base de datos MySQL desde contenedor local a otro contenedor que simula RDS.

Solución

```
# Origen
docker run -d --name mysql-origen -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=ventas -p
    3306:3306 mysql:8
# Destino
docker run -d --name mysql-destino -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=ventas -p
    3307:3306 mysql:8
# Dump y restore
mysqldump -h127.0.0.1 -P3306 -uroot -proot ventas > ventas.sql
mysql -h127.0.0.1 -P3307 -uroot -proot ventas < ventas.sql
```

Ejercicio 5: Configurar HAProxy para balancear entre dos réplicas PostgreSQL

Enunciado: Usar HAProxy para balancear conexiones de solo lectura entre dos réplicas.

Solución

```
# Crear red y conectar réplicas
docker network create ha-net
# Ejecutar HAProxy con configuración (ver diapositiva anterior)
# En cada réplica, crear usuario haproxy
docker exec -it pg-replica1 psql -U admin -c "CREATE USER haproxy;"
# Probar conexión
psql -h127.0.0.1 -p5432 -U admin -d testdb -c "SELECT inet_server_addr();"
```

Contenido de la sección

1. Introducción
2. Contenedores y Bases de Datos
3. Orquestación con Kubernetes
4. Estrategias de Alta Disponibilidad
5. Replicación en Motores Específicos
6. Balanceo de Carga Práctico
7. Despliegue en Nube vs On-Premise y Migración
8. Ejercicios Resueltos
9. Glosario

Términos Clave

Contenedor	Entorno aislado para ejecutar aplicaciones.
Docker	Plataforma de contenedores.
Volumen	Mecanismo de persistencia en Docker.
Kubernetes	Orquestador de contenedores.
StatefulSet	Recurso de Kubernetes para aplicaciones con estado.
Replicación	Copia de datos de un servidor a otro.
Réplica de lectura	Copia que acepta consultas de solo lectura.
Failover	Conmutación a nodo secundario ante fallo.
Split-brain	Dos nodos creyéndose primarios.
RTO	Recovery Time Objective.
RPO	Recovery Point Objective.
HAProxy	Balanceador TCP/HTTP.
ProxySQL	Proxy para MySQL.
pgpool-II	Middleware para PostgreSQL.
AWS RDS	Servicio de base de datos relacional gestionado.

¿Preguntas?

¡Gracias por su atención!